GRAPHIFY + AI MODEL
COMPLETE UPDATE & CLEAN CODING WORKFLOW

A clear guide showing when to use Claude Opus, Claude Sonnet, or Gemini Flash for your trading terminal.

ONE-LINE WORKFLOW

PLAN ALL UPDATES TOGETHER → BUILD ONE BY ONE → CLEAN ONE BY ONE → FINAL CHECK ONCE

CHOOSE YOUR AI MODEL FIRST

MODEL	USE IT FOR	AVOID AS FIRST CHOICE FOR
Claude Opus	Architecture, major concepts, difficult multi-file reasoning, complex bugs, trading calculations and major data-flow decisions.	Routine small edits when the task is already clear.
Claude Sonnet	Most normal coding: approved feature implementation, medium fixes, UI changes, testing, cleanup and documentation.	Very difficult architecture decisions with unclear scope.
Gemini Flash	Fast routine checks, simple edits, documentation checks, build/test reporting and clearly defined low-risk tasks.	Complex repository-wide reasoning or critical unclear calculations.

Important: Graphify and your project documentation are shared project resources. You can switch AI models; tell each model to read the existing documentation and analyze the current code before important changes.

PHASE 1 — PLAN ALL UPDATES TOGETHER

STEP 1 — UNDERSTAND THE MASTER REQUIREMENTS

GOAL: Understand all requested updates before touching code.

RECOMMENDED AI: Claude Opus for major or complex concepts. Claude Sonnet if requirements are already clear.

COMMAND FOR ANTI-GRAVITY

USE: CLAUDE OPUS

```
Here are all my requested updates:

UPDATE 1:
[Describe]

UPDATE 2:
[Describe]

Do not modify code yet.
Confirm your understanding of every update. Identify assumptions, unclear requirements, dependencies, and possible conflicts.
```

GREEN SIGN

Every update is understood and no code was changed.

STEP 2 — GRAPHIFY MASTER IMPACT ANALYSIS

GOAL: Map affected files and dependencies across all updates.

USE: CLAUDE OPUS

RECOMMENDED AI: Claude Opus for complex interconnected systems. Claude Sonnet Gemini Flash for a quick supporting review.

COMMAND FOR ANTI-GRAVITY

```
Do not modify code.

Read existing project documentation and use Graphify to analyze ALL requested updates.

For each update identify direct files, dependencies, shared files, critical files, conflicts, and safest implementation order.

Classify: REQUIRED | INDIRECTLY AFFECTED | REFERENCE ONLY | DO NOT MODIFY
```

GREEN SIGN

A clear master impact map and implementation order are available.

STEP 3 — CREATE THE MASTER PLAN

USE: CLAUDE OPUS

GOAL: Turn the analysis into a minimal implementation plan.

RECOMMENDED AI: Claude Opus for difficult tradeoffs. Claude Sonnet for a straightforward plan.

COMMAND FOR ANTI-GRAVITY

```
Do not modify code.

For each update provide:
Purpose
Files required
Why each file is required
Dependencies
Risk
Testing plan

Recommend the safest implementation order. Do not start coding.
```

GREEN SIGN

The plan is minimal, justified, and ready for review.

STEP 4 — PLAN REVIEW

GOAL: Remove unnecessary scope before implementation.

RECOMMENDED AI: Claude Sonnet. Gemini Flash can provide a second quick review. Use Claude Opus if there is disagreement on a critical decision.

COMMAND FOR ANTI-GRAVITY

USE: GEMINI FLASH

```
Do not modify code.

Review the master plan and classify every proposed file:
REQUIRED
POSSIBLE
NOT REQUIRED

Remove unrelated refactoring and provide the final approved implementation order.
```

GREEN SIGN

Only justified files remain.

PHASE 2 — BUILD & CLEAN ONE UPDATE AT A TIME

STEP 5 — IMPLEMENT ONE UPDATE

GOAL: Make only the approved current update.

RECOMMENDED AI: Claude Sonnet is the default. Use Claude Opus for critical calculations or complex multi-file logic. Gemini Flash for clearly defined simple changes.

COMMAND FOR ANTI-GRAVITY

USE: CLAUDE SONNET

```
Implement UPDATE [NUMBER] only using the approved plan.

Modify only files required for this update. Do not start the next update. Do not refactor unrelated code. Do not delete uncertain code.

If an additional file is required, STOP and explain why.

Report: FILES CHANGED / WHAT CHANGED / UNEXPECTED ISSUES
```

GREEN SIGN

The current update is complete and every changed file is justified.

STEP 6 — VERIFY & TEST

GOAL: Confirm the update works before cleanup.

RECOMMENDED AI: Claude Sonnet or Gemini Flash for routine verification. Claude Opus only when a critical failure needs deep investigation.

USE: GEMINI FLASH

COMMAND FOR ANTI-GRAVITY

```
Do not modify code.

Verify build, application startup, updated feature, direct dependencies, browser/server errors, and API/WebSocket connections if affected.

Report actual results using:
PASS | NOT TESTED | FAIL
```

GREEN SIGN

The feature works and direct dependencies were actually tested.

STEP 7 — FIX CONFIRMED BUGS

GOAL: Fix real failures using the smallest safe change.

RECOMMENDED AI: Claude Sonnet for normal bugs. Claude Opus for difficult root-caus obvious fixes.

USE: CLAUDE SONNET

COMMAND FOR ANTI-GRAVITY

```
Fix only confirmed failures.

Before changing code report:
BUG:
ROOT CAUSE:
MINIMUM FILES REQUIRED:

Use Graphify when dependency tracing is needed. Make the smallest safe fix and retest.
```

GREEN SIGN

The confirmed bug is fixed and retested.

STEP 8 — CLEANUP REVIEW

GOAL: Find dead, duplicate, temporary, or obsolete code created by this update.

RECOMMENDED AI: Gemini Flash or Claude Sonnet for routine cleanup. Claude Opus if deletion could affect complex trading or live-data logic.

COMMAND FOR ANTI-GRAVITY

USE: GEMINI FLASH

```
The current update is working.

Do not modify code yet.

Use Graphify to identify ONLY update-related unused imports, duplicate code, temporary debugging code, obsolete
code, unused variables/functions, or duplicate API/WebSocket calls.

For each finding report FILE / ISSUE / EVIDENCE / DEPENDENCY CHECK / SAFE TO REMOVE / RISK.
```

GREEN SIGN

A cleanup report exists and nothing was deleted without verification.

STEP 9 — CLEAN & RE-TEST

GOAL: Remove only verified items and prove nothing was broken.

RECOMMENDED AI: Claude Sonnet is the default. Gemini Flash for simple cleanup. Claude Opus for critical calculations or architecture.

COMMAND FOR ANTI-GRAVITY

USE: CLAUDE SONNET

```
Clean only approved SAFE TO CLEAN items.

Do not clean unrelated or uncertain code.

After cleanup: list files cleaned, run the build, retest the update, and retest direct dependencies.
```

GREEN SIGN

The update still works after cleanup.

PHASE 3 — FINAL CHECK ONCE

STEP 10 — FINAL REBUILD & VALIDATION

GOAL: Confirm all completed updates work together.

RECOMMENDED AI: Claude Sonnet for normal final validation. Claude Opus for major releases or critical-system changes. Gemini Flash for a supporting test/report review.

COMMAND FOR ANTI-GRAVITY

USE: CLAUDE SONNET

```
Do not add new features or refactor code.

After ALL updates are complete: run build, confirm application starts, test all completed updates together, check
critical panels, errors, and important API/WebSocket connections.

Report: BUILD / APPLICATION / UPDATES / DEPENDENCIES / NEW ERRORS / OVERALL STATUS

READY | REVIEW | NOT READY
```

GREEN SIGN

The project builds and completed updates work together.

STEP 11 — CHANGELOG & STOP

GOAL: Record the work and preserve the working state.

RECOMMENDED AI: Gemini Flash or Claude Sonnet. Do not spend Opus tokens on routine documentation.

COMMAND FOR ANTI-GRAVITY

USE: GEMINI FLASH

```
Do not modify application code.

Update only /docs/CHANGELOG.md with date, updates completed, files changed, cleanup performed, tests actually
performed, final result, and known limitations.

Then STOP. Do not begin unrelated improvements.
```

GREEN SIGN

The work is documented and the project is left in a stable state.

THE SIMPLE MODEL DECISION GUIDE

QUESTION	BEST FIRST CHOICE
Difficult, unclear, architectural, or critical trading logic?	CLAUDE OPUS
Normal approved feature, implementation, or medium bug fix?	CLAUDE SONNET
Routine, simple, clearly defined, or mainly checking/reporting?	GEMINI FLASH
Smaller model is confused or making unjustified changes?	ESCALATE TO OPUS
Opus has already created the difficult plan?	USE SONNET TO IMPLEMENT

BEST TOKEN-SAVING FORMULA

OPUS FOR DEEP THINKING → SONNET FOR MOST IMPLEMENTATION → FLASH FOR FAST ROUTINE CHECKS

Final rule: Choose the AI model based on the difficulty of the current step—not based on which model you used previously. Keep Graphify and your documentation updated so every model starts from the same project understanding.